What is Machine Learning?

By Adit Singh

The science of programming computers so that they can learn from data, understand the relations between inputs(data) and outputs such that it can make accurate predictions.
Its modern day magic.

Artificial intelligence > ***Machine Learning*** > Deep learning

AI refers to the ability of computer systems to perform tasks that typically require human intelligence, Machine Learning is a subset of this where it learns from data, this could be from various methods and models, such that its performance increases on a given task with experience.

Essentially it's a complex way to map various inputs to outputs that we want. For example, the inputs being an object's shape, weight, density, volume and colour, Which a model could map to a given set of Outputs which would be the models guess as to what that object may be; a book, ball, apple etc.
This is called Classification, there's various methods of Classification such as or where, given a certain number of sets and information on those sets, the model would form groups of average features (weight, colour etc) and then see where a new input with different values of features may fall based on this.

If the new input's weight is similar to a book's but its density is higher, the model may still classify it as a book if weight is a more influential feature. However, if density is weighted more heavily in the model's decision making, it might classify the object differently, perhaps as a denser item like a brick.

This begins to give us a good understanding of perceptrons and Neural networks, which you will soon discover aren't as convoluted as they sound, kind of.

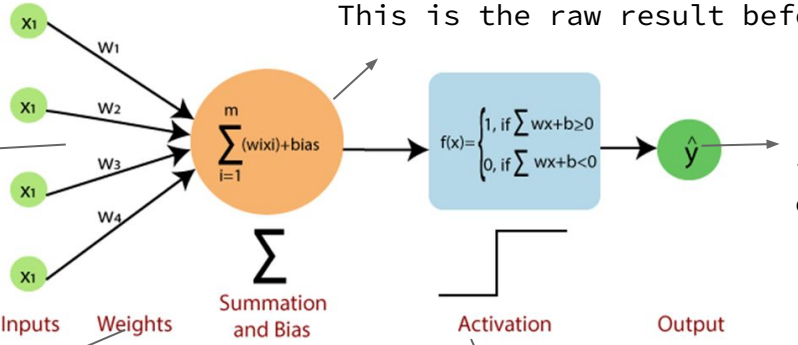
Perceptrons & Neural Networks

The goal is to replicate the human brain, and our brain is essentially layers of neurons that are each always sensing/computing something or the other. In this case, its output will be 1 or 0 based on the input features. Maybe it's detecting possible rain or if a user is likely to click on an ad for dogs.

A single Neuron (Perceptron)-

All weighted inputs are summed up:
bias helps shift the output to better fit the data.
This is the raw result before applying any decision.

These are the features or data values fed into the model



Final decision or prediction.
for binary classification, it's either 0 or 1.

Each input is multiplied by a weight these Weights determine the importance of each input. The model learns these weights during training.

The greater the weight of an input, the more it impacts the output.

The activation function decides the output based on the sum.
In this case, it's a step function:

- Output is 1 if the sum is ≥ 0
- Output is 0 if the sum is < 0

This determines whether the neuron "fires" or not.

Activation Function-

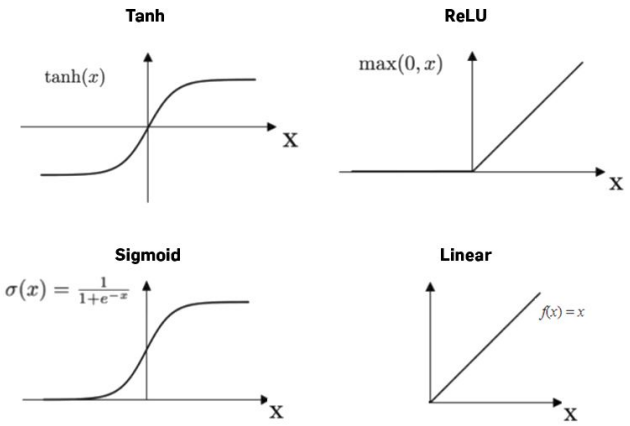
The activation function is what maps the calculated result of the summation + bias to a usable output, the function used is dependent on the purpose of the model.
The previously shown perceptron used a binary step function, meaning it either outputted a 0 or a 1 based on the summation + bias calculation. This binary output may be desired for a simple detector or validator, if an email is spam for example, true or false.

For much more complex purposes, different functions may be utilised in various ways. A sigmoid function is one often used, this is because it takes any real input and outputs a value between 0 and 1. This could be used if you wanted a percentage value of a certain input, maybe a probability of a given event happening based on various features.

These functions add non-linearities to the neural net, making it more effective and allowing it to learn much more complex patterns.

The sigmoid function in particular has a smooth gradient, making it a prime candidate to be used for gradient descent which we will explore in detail.

Here are some commonly used functions-



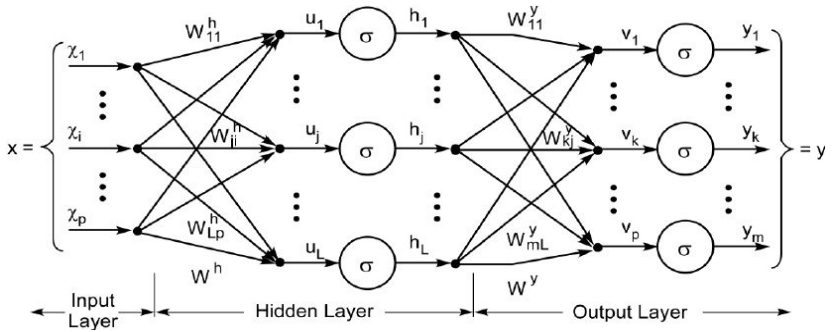
From Perceptrons to Neural Networks-

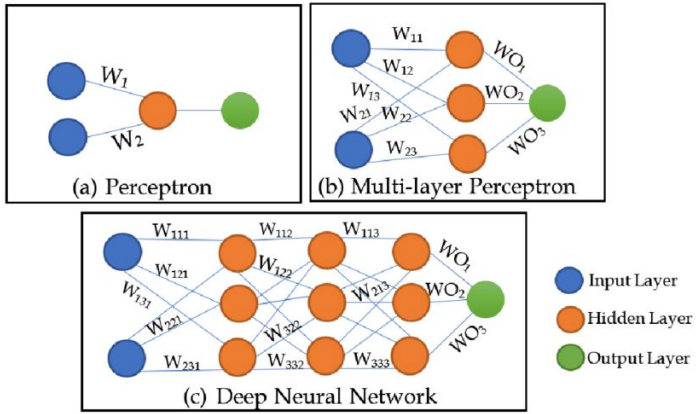
If the perceptron was a single artificial neuron, then a Neural Network is the entire brain. A neural network is a collection of many perceptrons arranged in layers. It’s designed to approximate complex, nonlinear functions.

A perceptron receives a signal (inputs/data) processes them (computes) and passes the signal on (pass output to next perceptron).

A neural network consists of MANY inputs, passed between multiple layers, where each layer has a matrix of weights for each set of inputs, they are able to handle much more complex computing. They are able to learn non-linear relationships in data. They have hidden layers of different weights

Thanks to our understanding of perceptrons, this doesn't look as bad.

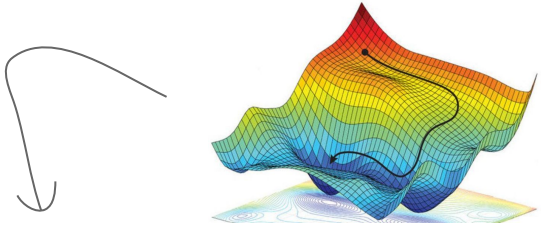




How does Machine Learning Actually Happen?

1. Make a prediction with the current model parameters (weights).
2. Compare Models Output (prediction) to actual known answer using a loss function (varies based on model used- if predicting numbers you may use the mean squared error).
3. Calculate how much each weight given to inputs contributed to the error, this is done through <u>backpropagation</u>- uses the chain rule to calculate the partial derivatives of the loss.

Imagine the Loss function as a hilly landscape, and each weight is a dimension across this landscape (like a direction), gets complicated for us to visualise after 3. The partial derivative dL/dwi, where wi is the weight of a specific input i.
The goal is to think "if i change this weight wi a little, how does the loss change (does the prediction get closer to actual value?)". We use these gradients to update the weights to essentially move down the hill of the loss function.

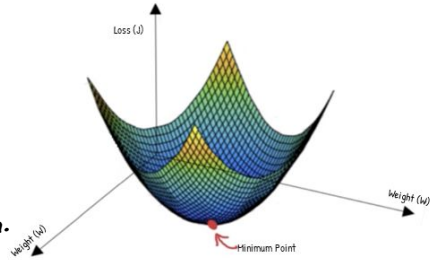


4. Update the weights using the gradients (from step 3) and a learning rate.
   This is where <u>gradient descent</u> happens. Each weight Wi is updated like this:
   $Wi \leftarrow Wi - n \times dL/dWi$

Here, n is the learning rate, a small number that controls how big each step is. This update moves the weights in the direction that reduces the loss, like rolling downhill toward a better prediction.

5. Repeat steps 1-4 for a batch or the whole dataset, (one epoch), repeat multiple times.
6. Check model performance on new data (Test Set) that wasn't used before. To see if it generalizes well to unseen data.
7. Tweak Parameters- (learning rate, activation function)
8. Stop Training when Loss no longer decreases or when it starts memorising data (test set)- overfitting.

BackPropagation and gradient descent??



Backpropagation is how a machine learning model learns from its mistakes.

Its super intuitive. Let's assume we have two inputs and two weights respectively, a bias and one output from this neuron.

Let's go back to our Perceptron, the single neuron. For up to i inputs, It has inputs Xi and Each input Xi has a Weight Wi which shows its impact/importance. These then get multiplied together and each product XiWi is added in the summation, finally adding a Bias b which allows the activation function to move to the left or right, shifting it such that it doesn't have to pass through the origin so that it can fit the data better. Let's call our actual known value Y and our prediction y.

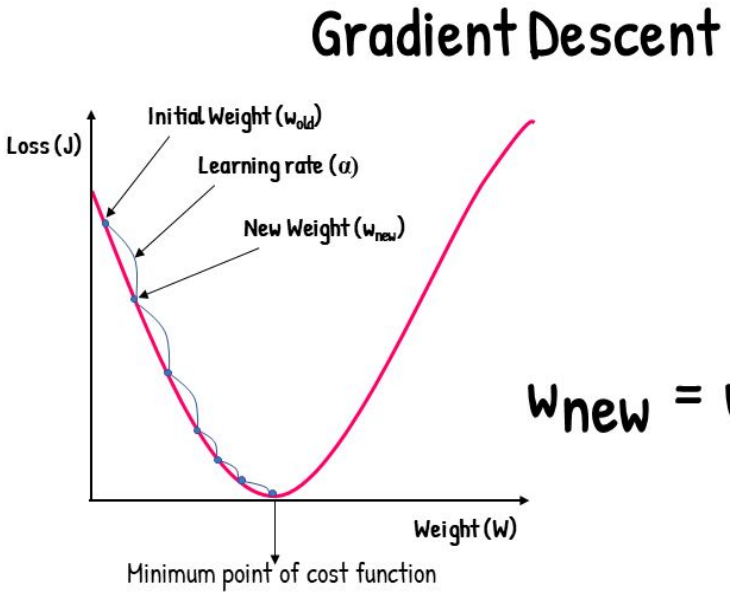
This final output, Z , is passed into the activation function, maybe the Sigmoid , $y = \sigma(Z)$, where $Z = (\text{Sum of } W_iX_i) + b$.
There are various Loss functions, if we use the Mean Squared error , $L = 1/2\,(Y-y)^2$, the ½ is there to simplify the derivative, convenience sake.

We want to see how this loss function changes with respect to the weights, because we want the loss (how far our prediction is from actual value) to be at a minimum. This is Backpropagation, finding the gradient of the loss function with respect to the weight of each feature.

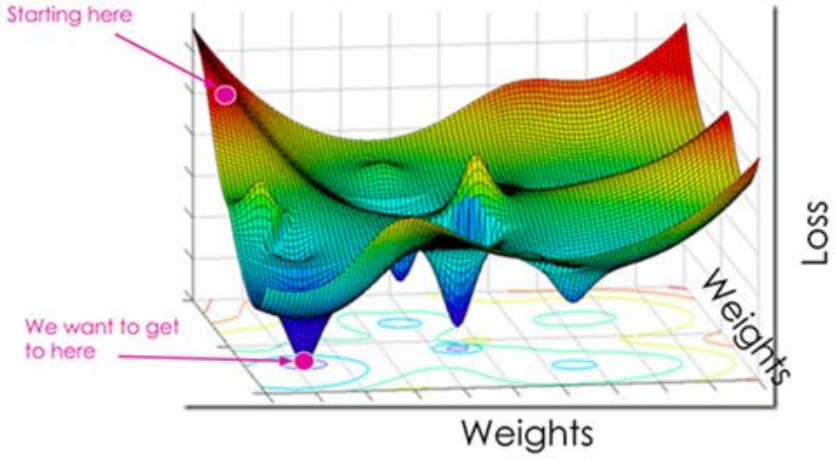
Once found, this gradient, multiplied by a learning rate(a step/jump), taken away from the original weight will give a new weight to feature i, such that the loss is less compared to before, meaning our prediction is much more accurate. Essentially we want the lowest loss possible given a weight i.

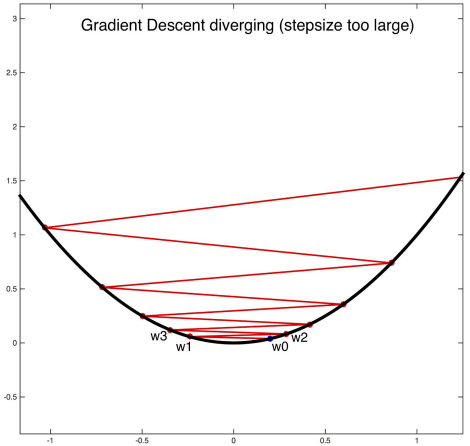
Once this global minima is found, the weights are not changed and the model is ready to be evaluated on test data. Its finished learning.

Visualising this- it sounds complicated but it really isn't



$$w_{new} = w_{old} - \alpha \frac{\delta J}{\delta w}$$





This is what happens if the learning rate is too big-
(The step to the next point on the loss function becomes too big and we diverge away from the point where the loss is the least, and if its too small it's going to take very long and may get stuck in a local minima- a small valley.)

BUT HOW??

The chain rule in ai?

This is how the gradient of the loss function is calculated-

$$\frac{dL}{dw_i} = \frac{dL}{dy} \times \frac{dy}{dz} \times \frac{dz}{dw_i}$$

How Loss (error) Changes with respect to y (prediction).

How $\sum_{j=0}^{i} (x_i w_i) + b$ Changes with respect to w_i.

How Loss Changes with Respect to w_i.

Impact of w_i on error produced by Model.

How our Output, Prediction, Changes with Respect to z

Makes much more sense now!

The uses of machine learning

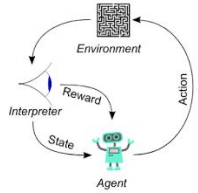
Machine learning had hundreds of possible application and is often more effective, simpler and efficient then the standard algorithmic approach to coding. There is a large number of scenarios in which machine learning can be advantageous to apply

- Problems in which large amount of fine tuning or long list of rules are required. Machine learning is often able to simplify code giving it better performance.
- Machine learning also performs very well in fluctuating environment such as those in financial setting where relevant data is short lived and constantly changing. This is because they can be maintained and updated far more easy than a fixed program meaning model rot is less of an issue.
- Getting insights about complex problems and large amounts of data, finding relationships previously hidden from us.

In use:

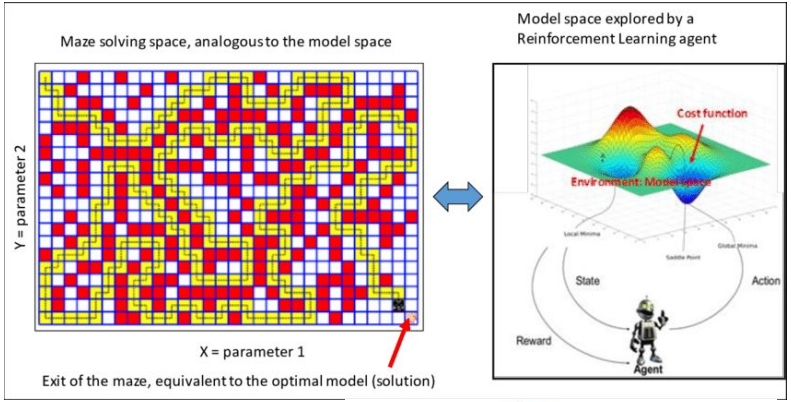
Facial Recognition
Detecting fraud financial transactions or Spam mails
Self driving cars
Predictive algorithms (stocks, housing market, traffic)
“Recommended”- Netflix, Social Media, Marketing (shows/ads)
Detecting Tumours in Scans
Compute and Visualise Math (Conjectures, Graphs, Geometry, Sequences etc)
Maps, Search Engines, Games etc
Robots(Reinforcement Learning)
It's everywhere.

What's reinforcement learning?

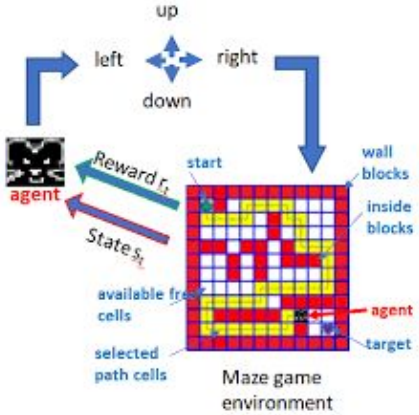


Reinforcement learning is a type of machine learning where an agent learns how to make decisions by interacting with its environment. Instead of relying on labeled data like in supervised learning, the agent tries different actions and learns from the outcomes. If it makes a good decision, it gets a reward; if it makes a bad one, it gets a penalty. Over time, the agent figures out which actions lead to the best results. Theres many such algorithms- Q-learning is one often used.

This approach is often used in areas like robotics, self-driving cars, and game playing, any situation where learning through trial and error makes sense. It’s kind of like how we learn from experience: try something, see what happens, and adjust next time.



However, reinforcement learning also comes with some significant challenges. Training an agent can be time-consuming and computationally expensive, especially when the environment is complex. It can also be unstable or inefficient if the reward signals are sparse or poorly designed. Despite these challenges, ongoing research and improvements in algorithms continue to make reinforcement learning a promising area of artificial intelligence.



A SIMPLE MODEL (Predicting life satisfaction from GDP)-

Importing necessary modules, essentially pre-written code to make it simple for us.

Plotting and Displaying the Graph itself, with the given increments along the axis.

Asking the model “what's the life satisfaction if GDP was 7074.19”

first.py ocr

work-experience > first.py > ...

```python
import numpy as np
import time as t
import matplotlib.pyplot as plt
import pandas as pd
import sklearn.linear_model



data_root = "https://github.com/ageron/data/raw/main/"
lifesat = pd.read_csv(data_root + "lifesat/lifesat.csv")
X = lifesat[["GDP per capita (USD)"]].values
y = lifesat[["Life satisfaction"]].values


lifesat.plot(kind='scatter', grid=True,
        x="GDP per capita (USD)", y="Life satisfaction")
plt.axis([23_500, 62_500, 4, 9])
plt.show()




model = sklearn.linear_model.LinearRegression()

model.fit(X,y)


X_new = [[7074.19]] #gdp per capita of brazil in 2020 (LS = 6.1)
print(model.predict(X_new))
```

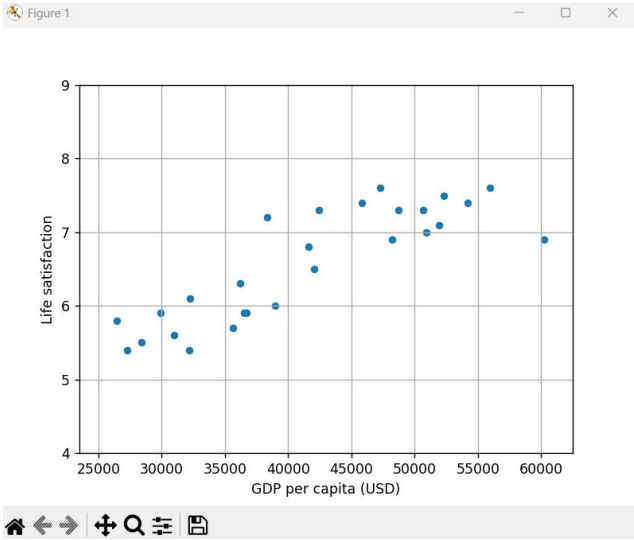
Reading the data necessary for the model.

Assigning GDP to x-axis and satisfaction to y-axis.

Applying a linear model to suit the data. A line that roughly matches up with the data that was just read.

What did it predict??

When fed the GDP per capita of Brazil in 2020, 7074.19 USD, the outputs were the following-



When using K-Neighbors Model-

```
PS C:\Users\User\OneDrive\Desktop\work-experience> & C:/Users/User
sktop/work-experience/first.py
[[5.56666667]]
```

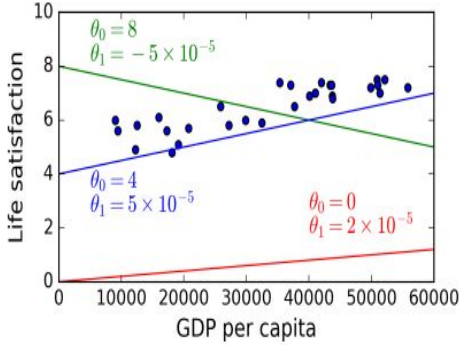
When using Linear Regression Model-

```
PS C:\Users\User\OneDrive\Desktop\work-experience
sktop/work-experience/first.py
[[4.22860167]]
```

When in actuality, the life satisfaction in Brazil was 6.1, i can imagine this may be to do with the fact that there are more things that contribute to happiness than money, a nice beach and good weather for example. **This showed us that these models were not good at generalization, they don't account for more complex features and are too simple.**

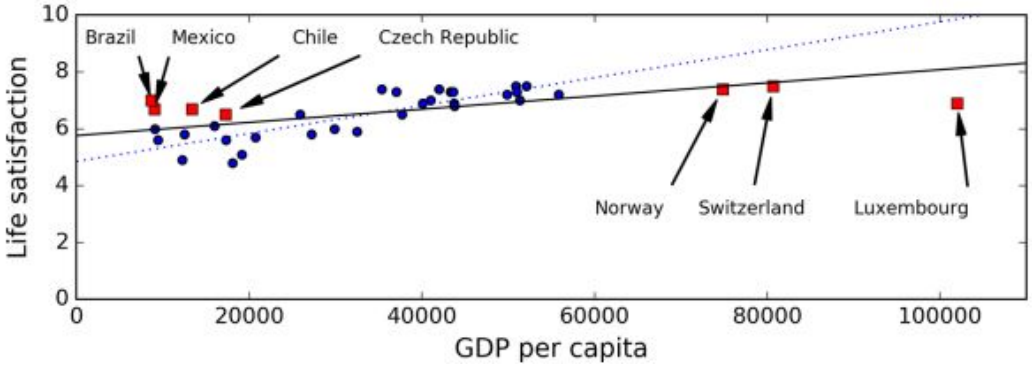
Problems with this model-

It assumes a perfectly linear relationship between GDP and Life satisfaction, this is due to the small data size of this model, and its simplicity, only having a single feature (GDP). Including weather, crime rates and Graduates per year would make it much more accurate. It assumed it must be a directly linear relationship.



Linear regression- "must be linear, has a y-intercept and gradient", tries them out and takes the one best suited to the data. This wasn't representative of our data however, due to our small training set.
By using a nonrepresentative training set, we trained a model that is unlikely to make accurate predictions, especially for very poor and very rich countries.

If we had a bigger training set, this is what we would see. Not only does adding a few missing countries significantly alter the model, but it makes it clear that such a simple linear model is probably never going to work well. It seems that very rich countries are not happier than moderately rich countries (in fact they seem unhappier), and conversely some poor countries seem happier than many rich countries.



Our Project- Estimating California's Housing Prices (a model)

The project that we have set out is to produce a simple model that is capable of estimating the price of a house in california given its address. We will be using the 1990 data set as it is easily accessible and provides a wide range of in detail data points which we can train the model on and test accuracy afterwards
Its
We have decided upon supervised learning for this as the available data set is labeled allowing us to feed exemplar data to our model (we know some houses and their prices). One testing metric that we could use is a test set followed by analysis of the generalization error as there is sample data for two sets.

We will be using batch learning as our method of feeding data to the model as we don't need the model to be based on current or new prices, it's simply based on old prices from 1990 and doesn't need to be online. We wish to predict house prices in 1990 based on multiple features such as population in district, proximity to ocean, occupancy, age of house etc.

The modules that we will be using include (Python):

- Pandas
- Numpy
- Matplotlib
- Sklearn
- jupyter

Downloading and accessing the data -

The dataset comes as a compressed .tgz file, the tarfile module is used to extract it. Everything's saved into a folder called datasets/housing to keep things organized. This setup helps make the project easier to share as anyone who runs the code will automatically get the same data without needing to download it themselves.

```python
#download and load the data
DOWNLOAD_ROOT = "https://raw.githubusercontent.com/ageron/handson-ml2/master/"
HOUSING_PATH = os.path.join("datasets", "housing")
HOUSING_URL = DOWNLOAD_ROOT + "datasets/housing/housing.tgz"

def fetch_housing_data(housing_url=HOUSING_URL, housing_path=HOUSING_PATH):
    os.makedirs(housing_path, exist_ok=True)
    tgz_path = os.path.join(housing_path, "housing.tgz")
    urllib.request.urlretrieve(housing_url, tgz_path)
    housing_tgz = tarfile.open(tgz_path)
    housing_tgz.extractall(path=housing_path)
    housing_tgz.close()

def load_housing_data(housing_path=HOUSING_PATH):
    csv_path = os.path.join(housing_path, "housing.csv")
    return pd.read_csv(csv_path)

#fetch and load the data
fetch_housing_data()
housing = load_housing_data()

#explore the data
print(housing.head())
print(housing.info())
print(housing.describe())
```

Downloads and extracts the dataset from the URL into the specified directory

The dataset comes as a compressed .tgz file, the tarfile module is used to extract it.

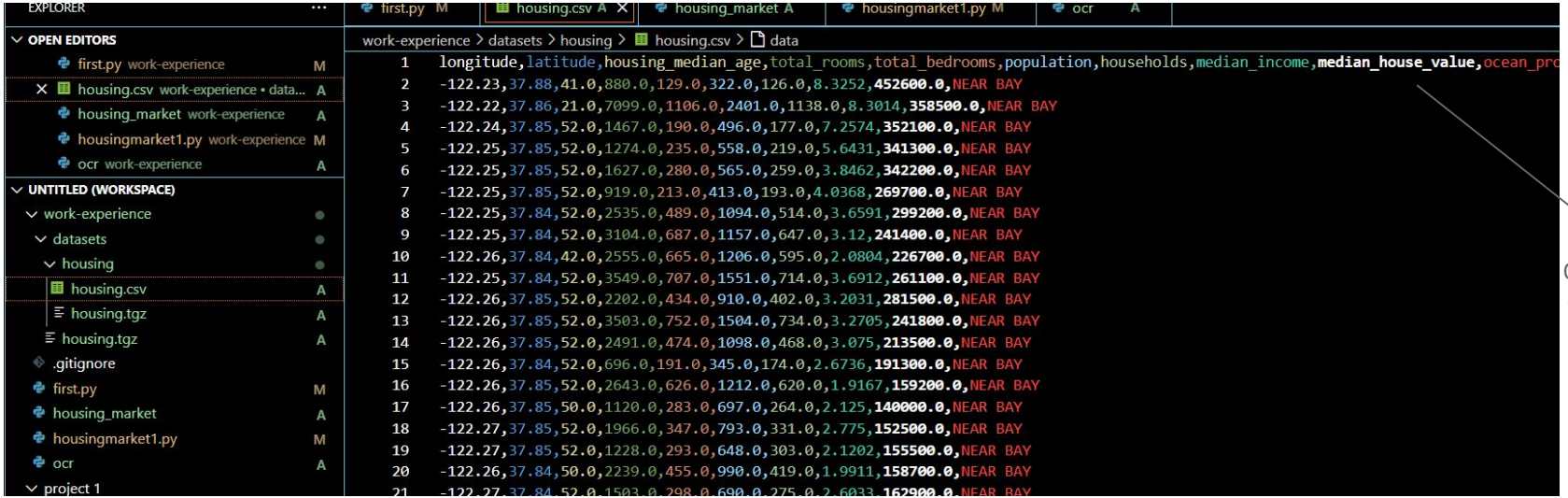
In this section the whole dataset is being downloaded using urllib.request to fetch the data from a URL and save it locally so it doesn't have to be done manually each time.

Downloads and loads the dataset.

Prints summary statistics, Data types and missing values

Loads the extracted csv file into pandas dataframe for analysis.

How does our data look?



Our label (actual value)

First row are just the column identifiers we used as feature names.

This is the product of running the load_housing_date() function, once run for the first time, it isn't needed again as the data is now stored on a local file.

Looking at this, we can see that ocean_proximity is the only feature that's not numerical, we adjust for this later in the model.

Various graphs could be plotted to visualise multi-feature relationships in this dataset.

Importing the libraries-

first.py M | housing.csv A | housing_market A | housingmarket1.py M

work-experience > housingmarket1.py > ...

```python
#import modules needed
import os
import tarfile
import urllib.request
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split, StratifiedShuf
from sklearn.preprocessing import StandardScaler, OneHotEncoder
from sklearn.impute import SimpleImputer
from sklearn.compose import ColumnTransformer
from sklearn.pipeline import Pipeline
from sklearn.linear_model import LinearRegression
from sklearn.tree import DecisionTreeRegressor
from sklearn.ensemble import RandomForestRegressor
from sklearn.metrics import mean_squared_error
from sklearn.model_selection import cross_val_score, GridSearchCV
from sklearn.base import BaseEstimator, TransformerMixin
```

Here, various libraries and modules used throughout the program are being imported and installed
These modules are used for:
Data handling (pandas, numpy)
File handling (os, tarfile, urllib)
Visualization (matplotlib)
Machine learning models (sklearn.linear_model, sklearn.tree)
Data preprocessing and transformation (pipelines, imputers, encoders)
Evaluation and tuning (cross-validation, grid search)
Libraries are prewritten code we can access to save time and effort.

Combining features-

This class performs feature engineering, our data currently has columns which are not very useful by themselves such as total rooms, total bedrooms, population and households in each district.

Although these values are relevant to our data there is a smarter way to implement them such that the model can learn quicker.

New Attributes are added to the data such as-

Rooms_per_household = total_rooms/ total_bedrooms

Population_per_household = total_population / total_households

Bedrooms_per_room = total_bedrooms / total_rooms

```python

#column index constants (for housing_num DataFrame)
rooms_ix, bedrooms_ix, population_ix, households_ix = 3, 4, 5, 6

class CombinedAttributesAdder(BaseEstimator, TransformerMixin):
    def __init__(self, add_bedrooms_per_room=True):
        self.add_bedrooms_per_room = add_bedrooms_per_room
    def fit(self, X, y=None):
        return self
    def transform(self, X):
        rooms_per_household = X[:, rooms_ix] / X[:, households_ix]
        population_per_household = X[:, population_ix] / X[:, households_ix]
        if self.add_bedrooms_per_room:
            bedrooms_per_room = X[:, bedrooms_ix] / X[:, rooms_ix]
            return np.c_[X, rooms_per_household, population_per_household, bedrooms_per_room]
        else:
            return np.c_[X, rooms_per_household, population_per_household]
```

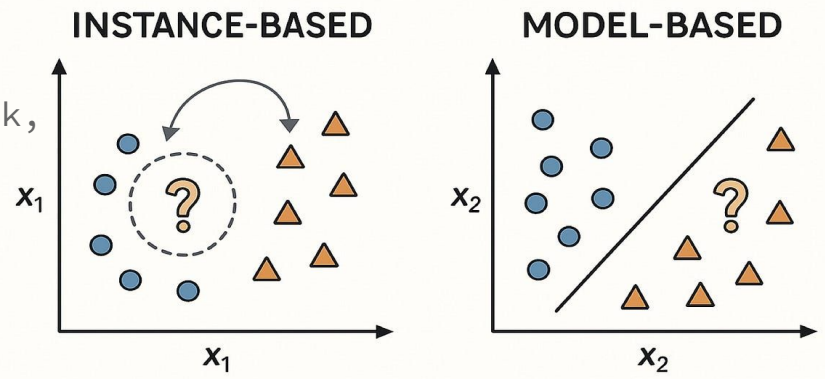
The types of learning in ML

Instance-based learning and model-based learning are the two main approaches in machine learning, each with its own strengths and weaknesses.

Instance-based learning stores all the training data and makes predictions by comparing new inputs to these stored examples. It doesn’t build a model, meaning it can adapt quickly to new data however it can be slow when making predictions because it searches through all instances.

On the other hand, model-based learning creates a general model from the training data, like a decision tree or neural network, which summarizes the data patterns.
This allows faster predictions since the model applies learned rules rather than searching data. However model-based methods may struggle if the model doesn’t capture the data well.

Both methods aim to predict outcomes accurately but differ mainly in how they use the data: instance-based relies on memorizing examples, while model-based focuses on learning underlying patterns.

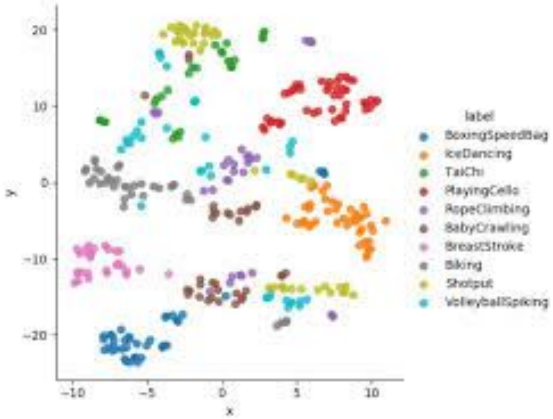


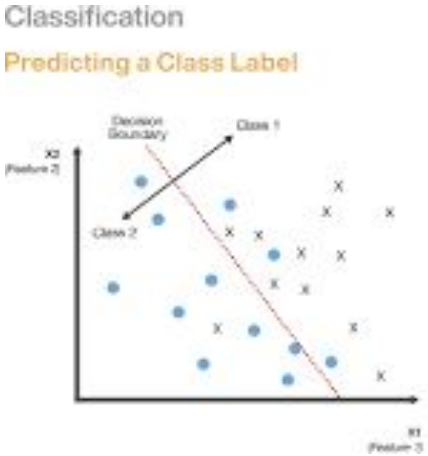
Instance-based learning involves keeping and comparing new data to existing data to make predictions. In contrast, model-based learning creates a representation of the data, such as a line or a curve, and uses this model to make predictions about new data.

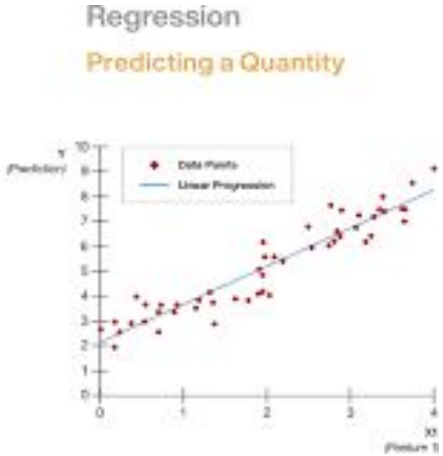
Training supervision–

The supervision applied to ai when training them refers to the methods through which the data set is fed to the model. There are many different types of supervisor that can be applied when training a model each having their own benefits and drawbacks. Types of supervision include:

- Supervised learning – In this form of supervision the training set that is fed to the algorithm includes the desired solutions. An example of this is classification task which are often performed by spam filters which would associate feature that correlate to a spam email and flag email that it identified as suspicious
- Unsupervised learning – Contrasting supervised learning, unsupervised learning has no labeled data fed into the model. For example clustering algorithms find correlation and groupsing unspecific to any model answer fed by the user, instead clustering example instances with similar processes together
- Semi-supervised learning – Some algorithms are able to deal with example sets that are only partly labeled which is useful as labeling data is expensive and time consuming to do.
- Self-supervised learning – This involves generating a fully labeled data set from a unabled one.





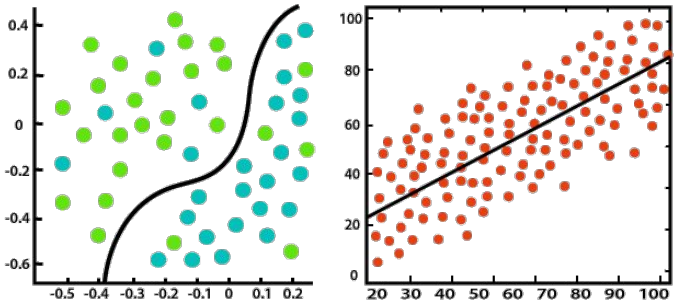


Our model- Regression vs classification

For models that want to output continuous data such as prices, time, temperature etc, Regression is typically used instead of classification as our goal isn't to group districts with similar prices into classes, but to output a prediction of what the actual price itself may be. This requires Regression to find the best-fit curve or line that minimises the error between actual and predicted values. This is usually done by minimising Mean Squared Error (MSE) or similar loss functions.

Conversely classification predict discrete categories or classes which is not the aim of the model since instead of trying to group together houses of similar price, location or otherwise we are trying to predict the most likely house price given these factors

The end product of our model should therefore be a graph similar to the regression graph shown on the right where a line of best fit is produced which predicts the most likely outcomes for the given parameters that the user enters.



Classification Regression

Making our data ready for model training-

```python
#separate features and labels
housing = strat_train_set.drop("median_house_value", axis=1)
housing_labels = strat_train_set["median_house_value"].copy()

#data preprocessing pipeline
num_pipeline = Pipeline([
    ('imputer', SimpleImputer(strategy="median")),
    ('attribs_adder', CombinedAttributesAdder()),
    ('std_scaler', StandardScaler()),
])

housing_num = housing.drop("ocean_proximity", axis=1)
num_attribs = list(housing_num)
cat_attribs = ["ocean_proximity"]



full_pipeline = ColumnTransformer([
    ("num", num_pipeline, num_attribs),
    ("cat", OneHotEncoder(), cat_attribs),
])

housing_prepared = full_pipeline.fit_transform(housing)
```

Contains all features, unlabeled

Contains Label (actual value) we want to predict.

1. Fills missing values with the median.
2. Adds engineered features.
3. Standardizes numerical attributes.

Separates numerical and categorical columns.

Applies num_pipeline to numerical features and converts ocean_proximity into binary indicator columns.

```python
#train models
#linear Regression
lin_reg = LinearRegression()
lin_reg.fit(housing_prepared, housing_labels)

#Decision Tree
tree_reg = DecisionTreeRegressor(random_state=42)
tree_reg.fit(housing_prepared, housing_labels)

#Random Forest
forest_reg = RandomForestRegressor(random_state=42)
forest_reg.fit(housing_prepared, housing_labels)
```

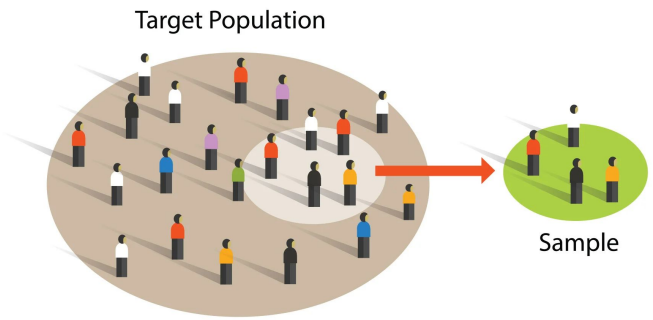
Here, the three different models are being trained:
-Linear Regression (baseline)
-Decision Tree (non-linear)
-Random Forest (ensemble), Bunch of decision trees together.
(only on the training set)

Issues with training data

There are many different possible issues that can occur when feeding data into a model or when trying to produce, label or find usable data sets for training

Such issue is sampling bias which is often produced as a result of how the data was collected. An example is nonresponse bias which result from people who don't care about a subject or issue not responding to surveys which can lead to a disproportionate representation of individuals that have more passionate opinions on the subject the polls about.

Irrelevant features within the data are also detrimental to the models progress. To much irrelevance within the data will result in an ineffective model. For this reason it is important to practice feature selection (picking the important features), feature extraction (combining features together to form new more relevant ones), creating new features by combining new data- this in general is referred to as Feature engineering.



Features and data used to train the model is vastly important in the models success and accuracy, quite reasonably so. If a baby is brought up being told an apple is actually called an orange it might be proven wrong later in life, leading to shame and embarrassment. This is exactly what happens, pretty much, if a model is trained on inaccurate data.

Infact, the accuracy and number of data the model is trained on is more important to decreasing accuracy loss than the model itself.

Stratified sampling and training splits-

We have used stratified sampling to ensure that the model performs well across all types of data in the dataset as well as ensuring that the model actually sees all the type of data during its training

```python
#create income categories for stratified sampling
housing["income_cat"] = pd.cut(housing["median_income"],
                               bins=[0., 1.5, 3.0, 4.5, 6., np.inf],
                               labels=[1, 2, 3, 4, 5])

#stratified split
split = StratifiedShuffleSplit(n_splits=1, test_size=0.2, random_state=42)
for train_index, test_index in split.split(housing, housing["income_cat"]):
    strat_train_set = housing.loc[train_index]
    strat_test_set = housing.loc[test_index]

#remove income_cat attribute
for set_ in (strat_train_set, strat_test_set):
    set_.drop("income_cat", axis=1, inplace=True)
```

Creates an income category feature for stratified sampling and helps ensure train/test sets have similar income distributions.
We don't want either set to have more houses of a certain type than the other.

Performs stratified splitting based on income categories. Ensures representative sampling in training and test sets.

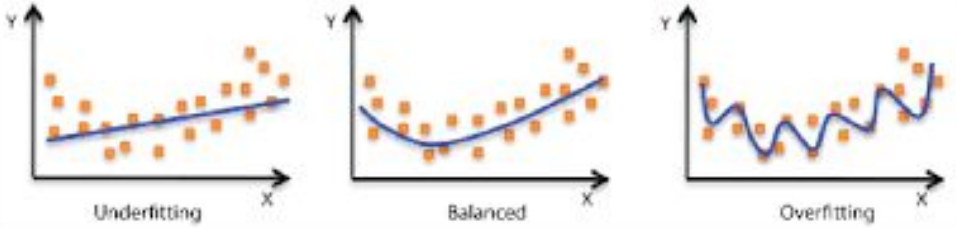
Removes the temporary income_cat column since it was only needed for splitting the data into sets.

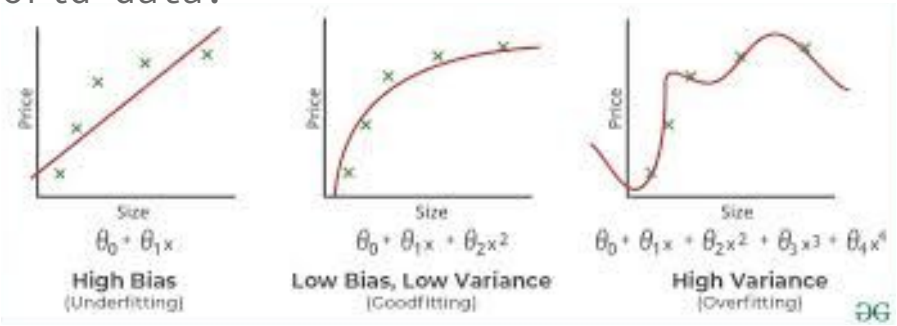
By using stratified sampling we will also get improved performance in cross-validation giving you more consistent and reliable model evaluation across folds. In addition to this it should be beneficial to the model's ability to generalize all key segments of the population, it’s more likely to generalize well to new, unseen data.

Overfitting vs underfitting data

Overfitting and underfitting are two common problems you can run into when training a machine learning model. Overfitting happens when the model learns the training data too well,even the random noise and small details,so it struggles to perform well on new data. It’s like memorizing answers for a test rather than understanding the material.

On the other hand, underfitting is when the model is too simple and doesn’t learn enough from the data, missing important patterns and relationships. That’s like trying to explain something complicated with an overly basic explanation, like when we used a simple linear model earlier on our life satisfaction model. The goal is to find a balance, where the model learns just enough to do well on both training and real-world data.





Choosing a Model and Tuning hyperparameters-

```python
#finetune the ranndom forest model
param_grid = [
    {'n_estimators': [3, 10, 30], 'max_features': [2, 4, 6, 8]},
    {'bootstrap': [False], 'n_estimators': [3, 10], 'max_features': [2, 3, 4]},
]

forest_reg = RandomForestRegressor(random_state=42)
grid_search = GridSearchCV(forest_reg, param_grid, cv=5,
                           scoring='neg_mean_squared_error',
                           return_train_score=True)
grid_search.fit(housing_prepared, housing_labels)

print("\nBest parameters:", grid_search.best_params_)
print("Best score:", np.sqrt(-grid_search.best_score_))

#get the best model
final_model = grid_search.best_estimator_

#evaluate on test set
X_test = strat_test_set.drop("median_house_value", axis=1)
y_test = strat_test_set["median_house_value"].copy()

X_test_prepared = full_pipeline.transform(X_test)
final_predictions = final_model.predict(X_test_prepared)

final_mse = mean_squared_error(y_test, final_predictions)
final_rmse = np.sqrt(final_mse)

print(f"\nFinal RMSE on test set: {final_rmse}")
```

This defines a grid of hyperparameters to search through n_estimators, max_features, and bootstrap are all being tuned.
This is done to avoid over/under fitting.

Here a new random forest regression model is being created, so that it can then be tuned.

GridSearchCV will search through all combinations in param_grid using 5-fold cross validation (cv=5).

scoring='neg_mean_squared_error' uses MSE testing as its metric.

return_train_score=True saves the training scores.

This runs the full grid search:

Trains 18 different models (one per param combo), Evaluates each with 5 fold cv and finds the best parameter combination based on CV score.

Retrieves best performing model.

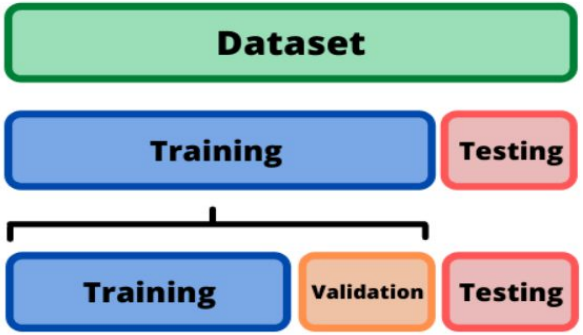
Evaluate on unseen data and give a final error value.

Testing and Validating a Model

Once you model has been finalised and trained with your given data it's necessary to test it and validate its effectiveness. There's many different methods that you could take to glean and objectively look at your models performance each having there own unique advantages and disadvantages

The error rate on this test set can be measured and is referred to as the generalization error of your model. By evaluating your model on a test set you can attain a general idea of how it will perform on new unseen data sets.

One possible method would be to place your model into production using real world data to test it. However if your model is wildly off or very poor in quality then this can end up being very annoying or unhelpful for your customer. Thus two sets of data are often produced. The Training set and Test set.



Other measure of performance include root mean square error (RMSE) and mean absolute error (MAE) both of which are methods to measure the distance between the prediction vectors and vectors of the target values. These methods both allow you to assign numerical values to the error between the predicted and actual result which is useful as it allows further calculations to be performed making it easier to determine which models are most effective.

$$\mathrm{MAE}(\mathbf{X}, h) = \frac{1}{m}\sum_{i=1}^{m}\left|h\left(\mathbf{x}^{(i)}\right) - y^{(i)}\right|$$

$$\mathrm{RMSE}(\mathbf{X}, h) = \sqrt{\frac{1}{m}\sum_{i=1}^{m}\left(h\left(\mathbf{x}^{(i)}\right) - y^{(i)}\right)^2}$$

Evaluating the Models-

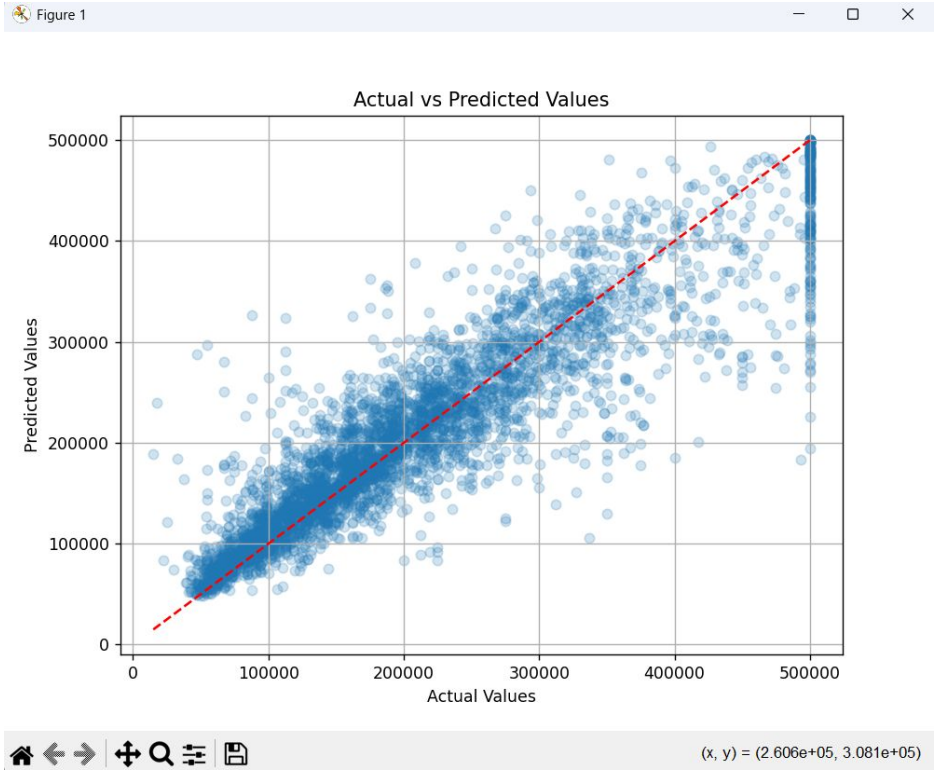
Since we trained 3 regression models all on the exact same training set, we can evaluate their accuracy on the same test set, meaning we only had to split data once and we can evaluate each model fairly.
RMSE (root mean square error) was used.

```python

#evaluate models using cross-validation
def display_scores(scores):
    print("Scores:", scores)
    print("Mean:", scores.mean())
    print("Standard deviation:", scores.std())

print("Linear Regression:")
scores = cross_val_score(lin_reg, housing_prepared, housing_labels,
                         scoring="neg_mean_squared_error", cv=10)
lin_rmse_scores = np.sqrt(-scores)
display_scores(lin_rmse_scores)

print("\nDecision Tree:")
scores = cross_val_score(tree_reg, housing_prepared, housing_labels,
                         scoring="neg_mean_squared_error", cv=10)
tree_rmse_scores = np.sqrt(-scores)
display_scores(tree_rmse_scores)

print("\nRandom Forest:")
scores = cross_val_score(forest_reg, housing_prepared, housing_labels,
                         scoring="neg_mean_squared_error", cv=10)
forest_rmse_scores = np.sqrt(-scores)
display_scores(forest_rmse_scores)
```

```
Linear Regression:
Scores: [71762.76364394 64114.99166359 67771.17124356 68635.19072082
 66846.14089488 72528.03725385 73997.08050233 68802.33629334
 66443.28836884 70139.79923956]
Mean: 69104.07998247063
Standard deviation: 2880.3282098180584

Decision Tree:
Scores: [72831.45749112 69973.18438322 69528.56551415 72517.78229792
 69145.50006909 79094.74123727 68960.045444   73344.50225684
 69826.02473916 71077.09753998]
Mean: 71629.89009727491
Standard deviation: 2914.035468468928

Random Forest:
Scores: [51559.63379638 48737.57100062 47210.51269766 51875.21247297
 47577.50470123 51863.27467888 52746.34645573 50065.1762751
 48664.66818196 54055.90894609]
Mean: 50435.58092066179
Standard deviation: 2203.3381412764606

Best parameters: {'max_features': 8, 'n_estimators': 30}
Best score: 49898.98913455217

Final RMSE on test set: 47873.26095812988
```

Lowest score means greatest accuracy. Since our Random Forest model had the lowest RMSE, we decide to use this model and train it on the test set. Pretty logical.

Our final model-



If our Model was perfect, there would be no variance and all data points would lie exactly on the dotted line, as every predicted value would be equal to the actual value, a y=x graph in the positive quadrant. There is a quite high deviation across the predicted and actual values, where the model overestimates for low priced houses and underestimates for high priced houses.

Although there are some discrepancies, this graph shows us our model is generally quite accurate, for the limited data it was trained on.

Using RMSE, we had a Final RMSE on test set of 47873.26095812988.
Essentially our predictions are roughly off by £50,000, not terrible, not amazing either.

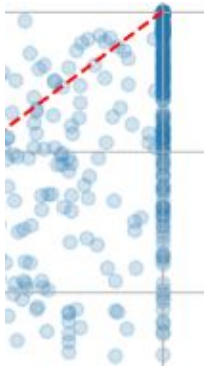
```
#scatter plot of actual vs predicted values
plt.figure(figsize=(8,6))
plt.scatter(y_test, final_predictions, alpha=0.2)
plt.plot([y_test.min(), y_test.max()], [y_test.min(), y_test.max()], 'r--')
plt.xlabel("Actual Values")
plt.ylabel("Predicted Values")
plt.title("Actual vs Predicted Values")
plt.grid(True)
plt.show()
```

LOOKING DEEPER INTO OUR MODEL-

As mentioned previously, there are different ways to measure a models performance, we used root mean squared error (RMSE) for ours, as it's typically a good measure of how far predictions are from actual values in regression tasks such as this.

RMSE was used to evaluate the performance of the Random Forest model, providing an interpretable measure of prediction error in the same units as house prices. While the model performed well overall, the RMSE was affected by consistent over and under predictions at the lower and upper ends of the price spectrum, respectively, Random Forests are known to often regress extreme predictions toward the mean, this is because the final output is an average of all the trees, the model stays conservative- preferring to stay in the middle rather than predict too high or too low.

Additionally, our dataset was capped at 500,000 for Median house value which may have affected this, this explains the dotted vertical line of predictions at 500,000 where the model was least confident in its predictions. You can see that closer here-



This shows how relevant the data you are training is. (Very)

Visualising our data-

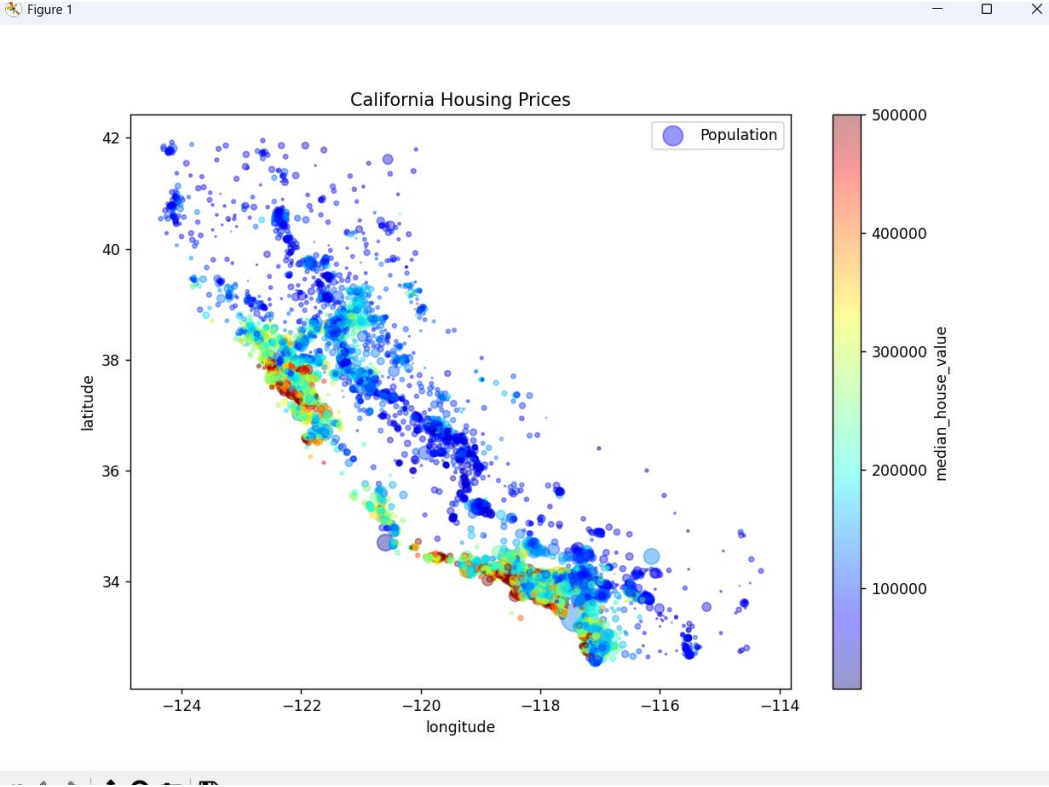
Red or yellow areas (e.g, around San Francisco and parts of LA) have higher median house values.

Blue/green areas (inland regions) have lower prices.

Dots near the coast tend to have warmer colors, supporting the idea that ocean proximity correlates with higher house prices.

Clusters of large blue dots might indicate densely populated but lower-income areas.

```python

housing = strat_train_set.copy()

#scatter plot geographically
housing.plot(kind="scatter", x="longitude", y="latitude", alpha=0.4,
             s=housing["population"]/100, label="Population",
             c="median_house_value", cmap=plt.get_cmap("jet"), colorbar=True,
             figsize=(10, 7), sharex=False)
plt.legend()
plt.title("California Housing Prices")
plt.show()
```



Feature Importance (Input Weights)-

```python

#feature importance
feature_importances = grid_search.best_estimator_.feature_importances_
extra_attribs = ["rooms_per_hhold", "pop_per_hhold", "bedrooms_per_room"]
cat_encoder = full_pipeline.named_transformers_["cat"]
cat_one_hot_attribs = list(cat_encoder.categories_[0])
attributes = num_attribs + extra_attribs + cat_one_hot_attribs
print("\nFeature importances:")
for importance, attribute in sorted(zip(feature_importances, attributes), reverse=True):
    print(f"{attribute}: {importance}")

```

```
Feature importances:
median_income: 0.3790092248170967
INLAND: 0.16570630316895876
pop_per_hhold: 0.10703132208204354
longitude: 0.06965425227942929
latitude: 0.0604213840080722
rooms_per_hhold: 0.05477891501828372
bedrooms_per_room: 0.048203121338269206
housing_median_age: 0.04218822024391753
population: 0.015849114744428634
total_bedrooms: 0.015554529490469328
total_rooms: 0.01524505568840977
households: 0.014934655161887774
<1H OCEAN: 0.006792660074259966
NEAR OCEAN: 0.0030281610628962747
NEAR BAY: 0.0015247327555504937
ISLAND: 7.834806602687504e-05
```

After training, we extracted feature importances from the model to understand which variables were most influential, this helps explain what the model "looks at" when making predictions and validates that our feature engineering had an impact, good to know. These are the weights we discussed back when we were looking at perceptrons and neural networks, cool right. These are our Wi, our features are our Xi and our predictions are yi, our labels are Yi.

Top Features Identified(also shown by graph)-

- **median_income** (strongest predictor, most impact)
- **rooms_per_household** (engineered feature)
- **bedrooms_per_room** (engineered feature)
- **population_per_household**
- **Ocean proximity category** (one-hot encoded,e.g- 1000 may be for inland)

Doesn't seem so complicated anymore right? Well..

Ai and Machine Learning aren't the enigmas we make them make out to be, but the first of many steps into that sci-fi future we always see in movies, it's really fascinating how a block of machine we previously thought to be limited in its operations can now virtually do anything given the correct model and instructions. Their limit is what we tell it to be, yet we are still not at the level of artificial intelligence you typically think of when you hear “AI”.

You are now one step closer to coding your own AI, but with this comes deeper issues concerning ethics.
Would you be stealing someone's job? Can AI steal everyone's job? Even coders and software engineers??

Although we haven't yet reached conscious sentient robots in society, it's a definite possibility in the visible future, starting out as helpers for those who need it, growing to rule humanity with an iron fist, quite literally.

To prevent this, we need to keep the mathematicians and computer scientists alive and well, and pay them well. To think that someday you may be humanity's only hope against salvation, and it all started with some machine learning models.

Or we can insure that reality comes to pass, how might we go about that, should AI work be stopped?
Is it increasing productiving or complacency? How does it affect the coming generations and the way they are taught?
AI is incredibly versatile and interesting, we hope this presentation increased your curiosity, if by a millimetre.

This is the link if you would like to play around with and test our Model- Model Visualiser (ui by lovable ai)

-Made by Adit Singh